



COMMUNITY SERVICE REQUEST MANAGEMENT PLATFORM

CivicConnect

PED v2.0 and Baseline Sign-off

Checkpoint	Status
Project	CivicConnect
Baseline Type	Architecture, Technology and Initial Design Baseline
Version	V2.0
Date	30 September 2026
Scope Reviewed	YES
Requirements / Traceability checked	YES
Risk review completed	YES
Repository / governance controls checked	YES
Outcome	ACCEPTED



30 SEPTEMBER 2026

BY

MATAMELA DAVHANA 600757

MKHANYISI MQADI 603000

JININY NKOMO 575936

1. Executive decision summary	3
2. Milestone 1 baseline review	3
3. Architecture-significant requirements and quality drivers	4
3.1 Quality scenarios used for architecture evaluation	5
4. Architecture alternatives and selection	5
4.1 Alternatives	5
5. Selected architecture	7
5.1 System context	7
5.2 Container and deployment view	7
5.3 Internal module view.....	8
5.4 Integrity-critical request creation	9
5.5 Architecture rules	9
6. Technology stack decision	9
6.1 Stack fit and proportionality	9
6.2 Version and compatibility control	10
7. Architecture Decision Records.....	11
7.1 ADR-ARCH-001 - Select a layered modular monolith	11
7.2ADR-TECH-001 - Select React, Node.js/Express and PostgreSQL	11
7.3 ADR-MOD-001 - Enforce feature-aligned module boundaries	12
8. RTM evolution and evidence plan	12
8.1 End-to-end trace example: request creation	13
9. Risks, trade-offs and forward engineering.....	13
9.1 Deliberate deferments.....	14
10. Data Entities & Ownership	14
11. Initial Data Model (ERD)	15
12. ADR-M2-DATA-001: Persistence Model	16
Alternatives considered	16
Decision.....	16
Rationale	16
Integrity, concurrency and risk measures	16
Trade-offs	17

Later consequence	17
13. ADR-M2-TECH-002: Technology Stack Selection	18
Context.....	18
Alternatives considered	18
Decision.....	18
Rationale	18
Hosting decision and documented free-tier limits (CON-003).....	19
Trade-offs	19
Versions (confirmed, as scaffolded).....	19
Later consequence	19
14. RTM Updates - Data/Persistence & Technology Columns	20
15. Risk Register Additions	20
16. Deployment Compatibility (Section 5.8).....	20
17. Design Decisions.....	21
17.1 Design Problem 1: Request Type Handling	21
17.2 Design Problem 2: Status Change Notification Coupling	22
17.3 Engineering Decision Log (ADR).....	23
18. Deployment Compatibility	25
19. Appendix A AI Usage Register Entry	26

1. Executive decision summary

CivicConnect should be implemented as a layered modular monolith: a React browser interface communicates with one Node.js/Express application, which stores transactional data in PostgreSQL. The backend remains one deployable unit while enforcing internal boundaries for Identity and Access, Request Management, Workflow, Audit and History, Reporting, and Administration. This structure gives the three-person team clear responsibilities without introducing the network, deployment, observability and distributed-data overhead of microservices.

The decision directly answers the Milestone 1 deferment of architecture, database, framework and hosting choices. It also addresses the small-team, limited-schedule and low-cost constraints while preserving a deliberate path to later extraction if measured load, independent deployment needs or organisational growth justify it.

Layering separates presentation, business logic and infrastructure through defined interfaces, while a modular structure aims for high cohesion within modules and low coupling between them. These properties support maintainability, testability and controlled evolution (Sharma, Kumar and Agarwal, 2015; Skiada et al., 2018).

A monolith is proportionate for the current workload and team because single-machine evaluations show that a monolith can outperform microservices, while microservice boundaries add orchestration, data consistency and resource costs that may outweigh their benefits for smaller systems (Blinowski, Ojdowska and Przybyłek, 2022).

2. Milestone 1 baseline review

Milestone 1 established the stakeholder needs, scope, prioritised requirements, measurable acceptance criteria, constraints, risk register and initial traceability foundation. It deliberately deferred authentication, architecture, database, frameworks and hosting to Milestone 2. The architecture work below keeps the responsive web scope and does not reopen deferred external notifications or other excluded features.

M1 baseline item	Continuity into M2	M2 evolution / decision	Evidence status
RD-10 technology and architecture deferment	Must now be resolved through research and ADRs	Resolved by ADR-ARCH-001 and ADR-TECH-001	Research complete; team approval pending
CON-001: three-person team	Architecture must be understandable and operable by a small team	One backend deployable, one database, explicit internal modules	Proposed
CON-002/003: schedule and low cost	Avoid unnecessary infrastructure and coordination overhead	No microservices, broker or distributed datastore in M2	Proposed

CON-004: responsive web application	Preserve browser-based scope	React UI over HTTPS/JSON API	Proposed
CON-005: measurable quality	Quality claims must be testable	ASRs and architecture tests are defined below	Planned evidence
CON-006: security and privacy	Least privilege and data protection remain mandatory	Central authentication/authorisation middleware and RBAC policy	Proposed
CON-008: compare technology options	Selection requires evidence and trade-offs	Alternatives compared in Sections 4 and 6	Complete
RISK-001: unfamiliar stack	Learning curve can threaten delivery	Shared JavaScript runtime reduces language switching; PoCs gate confirmation	Open - mitigation planned
NFR-007/008: integrity and auditability	No partial records; complete actor/action/time history	PostgreSQL transactions and append-only audit path	Proposed
NFR-012: maintainability	Reviewed and checked changes	Module rules plus protected-main CI workflow	Planned evidence

3. Architecture-significant requirements and quality drivers

An architecture-significant requirement (ASR) is a requirement whose satisfaction materially changes structure, technology, boundaries or deployment. The quality drivers below were derived from the M1 FRs, NFRs, constraints and risks rather than treated as generic desirable qualities. Web systems need explicit attention to quality and reliable, rapid updates; architecture should therefore make those qualities visible and verifiable (Offutt, 2002; CivicConnect Team, 2026a).

ASR	Source and measurable expectation	Architectural response	Priority / trade-off
ASR-01 Security and privacy	NFR-001/002; 100% of unauthorised access attempts denied; no sensitive content in prohibited views, logs or errors	Central authentication middleware; role-policy service; least-privilege checks at API and service boundaries; log redaction	Critical. Extra checks are accepted over marginal latency.
ASR-02 Integrity	NFR-007; zero duplicate references; failed operations create no partial request or unmatched history	PostgreSQL transaction for request + initial history; uniqueness and foreign-key constraints; rollback on failure	Critical. Strong consistency is selected over eventual consistency.
ASR-03 Auditability	FR-017 and NFR-008; actor, action, record and time captured for all required events	Dedicated audit/history module; append-only writes in the same transaction where correctness requires it	Critical. Storage growth is accepted and managed through retention/indexing.
ASR-04 Performance and scale	NFR-003; 10,000 requests / 50 concurrent users; 95% core actions <2s; summaries <5s	Stateless API where practical; indexed queries; pagination; measured React bundle; database connection pool	High. Optimise after profiling, not by adding distributed services prematurely.

ASR-05 Maintainability and testability	NFR-009/012; each Must Have linked to a test; reviewed changes and automated checks	Layered modules; dependency rules; unit/service/API tests; CI quality gates	High. Some interface boilerplate is accepted to preserve boundaries.
ASR-06 Availability and recovery	NFR-010/011; 99% production availability; daily backups; RPO ≤24h; RTO ≤4h	Health endpoint; automated deployment rollback; scheduled backups and restore rehearsal	Should. Single instance is acceptable initially if monitoring and recovery evidence exist.
ASR-07 Accessibility and responsiveness	NFR-005/006; WCAG 2.2 AA; 360/768/1280px on named browsers	Reusable accessible React components; responsive layout; automated and manual accessibility checks	High. UI consistency takes priority over one-off page customisation.

Role-based access control associates privileges with roles and scales access-policy administration across many users. CivicConnect should therefore keep role and permission decisions in one policy component while still checking record-level context where a simple role is insufficient (Bacon, Moody and Yao, 2002).

Security must remain traceable from requirements into design, implementation and verification activities. A security mapping approach is useful because it converts broad security expectations into operational steps and auditable evidence (Lee et al., 2025).

3.1 Quality scenarios used for architecture evaluation

Driver	Stimulus and environment	Required response	Measure / evidence
Security	A community user requests a staff-only endpoint in normal operation	API rejects before business operation; no sensitive payload is returned	403 response; security test; redacted log entry
Integrity	Database failure occurs after the request insert but before initial history insert	Whole unit of work is rolled back	No request row and no orphan history row; integration test
Performance	50 concurrent users submit/search within a database of 10,000 requests	System remains responsive under defined workload	95th percentile <2s for core actions; load-test report
Maintainability	A new request category rule is introduced	Change stays inside the Request/Workflow modules and associated tests	No new cross-module dependency; CI passes
Recovery	Primary database data is lost after the daily backup	Service is restored within agreed objective	Restore completed ≤4h; data loss ≤24h; drill record
Accessibility	Keyboard-only user completes request submission at 360px viewport	All controls operable and errors announced	WCAG test record; browser/view evidence

4. Architecture alternatives and selection

4.1 Alternatives

Alternative A - Unstructured monolith. One application and database with no enforced module or layer boundaries. It is quick to start but encourages business logic in routes and direct data access across features. This increases architecture erosion and makes changes harder to isolate.

Alternative B - Layered modular monolith. One deployable application with explicit presentation, application/domain and persistence layers plus feature-aligned modules. It preserves simple deployment while allowing internal responsibilities to be tested and evolved independently.

Alternative C - Microservices. Identity, requests, workflow, audit and reporting become separately deployed services with network APIs and potentially independent storage. It offers release and scaling autonomy but adds failure modes, monitoring, deployment and data-consistency work.

Modularity is strengthened by high internal cohesion and limited dependencies between modules. Poor modularity is associated with architectural technical debt, so the selected approach needs enforceable boundaries rather than merely naming folders (Skiada et al., 2018).

Criterion	Weight	A: unstructured monolith	B: modular monolith	C: microservices
Fit for three-person team and schedule	20%	4 - simple but risky to maintain	5 - simple deployment with clear ownership	1 - high operational overhead
Security and privacy control	15%	3 - possible but easily scattered	5 - central policy with bounded modules	3 - more network/service surfaces
Integrity and auditability	15%	4 - one transaction possible	5 - one transaction plus dedicated audit module	2 - distributed consistency required
Maintainability and testability	20%	2 - boundaries erode	5 - cohesive modules and test seams	4 - isolated services, but contract testing needed
Performance at M1 target	10%	5 - no network hops	5 - no internal network hops	3 - serialization and network overhead
Scalability beyond initial target	10%	2 - coarse-grained scaling	4 - vertical/replica scaling; extraction path	5 - independent scaling
Deployment, observability and cost	10%	5 - one unit	5 - one unit and one DB	1 - multiple services and pipelines
Weighted result	100%	3.45 / 5	4.90 / 5 - selected	2.65 / 5

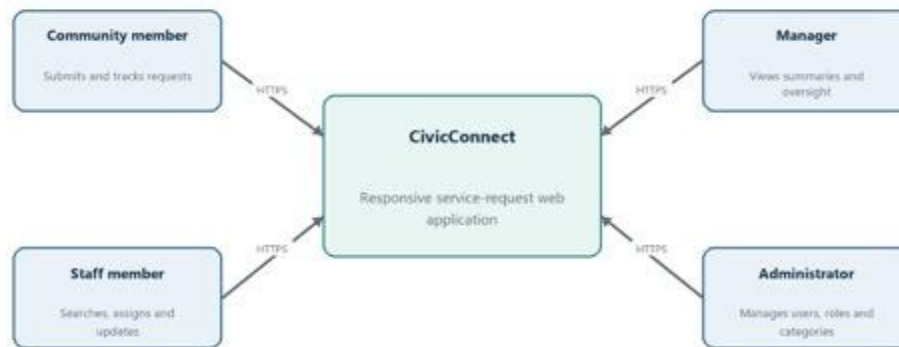
The score is a project decision aid, not a universal ranking. It reflects current CivicConnect constraints and the M1 workload. Empirical comparison found that a monolith on one machine outperformed microservices and that vertical scaling could be more cost-effective than horizontal scaling in the tested conditions. Microservice benefits should therefore be earned by an actual need for independent scaling or deployment (Blinowski, Ojdowska and Przybyłek, 2022).

Selected alternative: Alternative B - layered modular monolith. Alternative C is deliberately deferred, not prohibited. A later ADR may extract one module after load tests, change-frequency evidence and operational capacity justify the boundary.

5. Selected architecture

5.1 System context

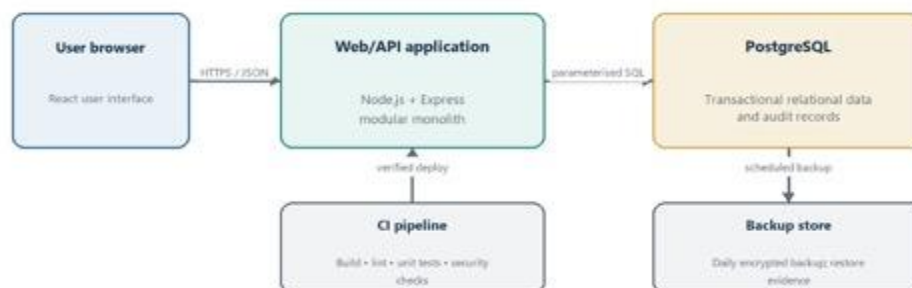
Figure 1. CivicConnect system context



The four user groups interact with the same responsive web application. External notification services remain outside the M2 baseline.

5.2 Container and deployment view

Figure 2. Proposed container and deployment view

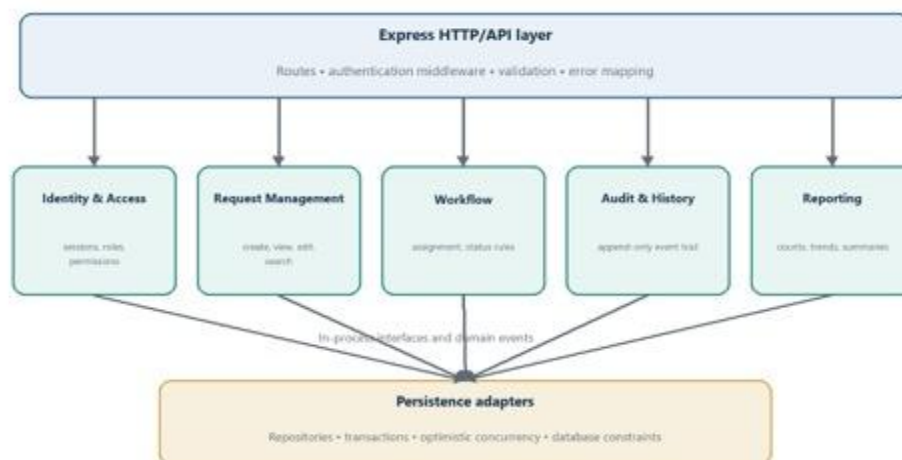


The proposed runtime has three primary containers: browser UI, one backend application and one relational database. CI and backup storage provide delivery and recovery controls.

The client-server and layered styles fit browser applications and separate the presentation, business and infrastructure responsibilities. Clear interfaces allow each layer to change with less impact on the others and support more focused debugging and maintenance (Sharma, Kumar and Agarwal, 2015).

5.3 Internal module view

Figure 3. Backend module boundaries inside one deployable application

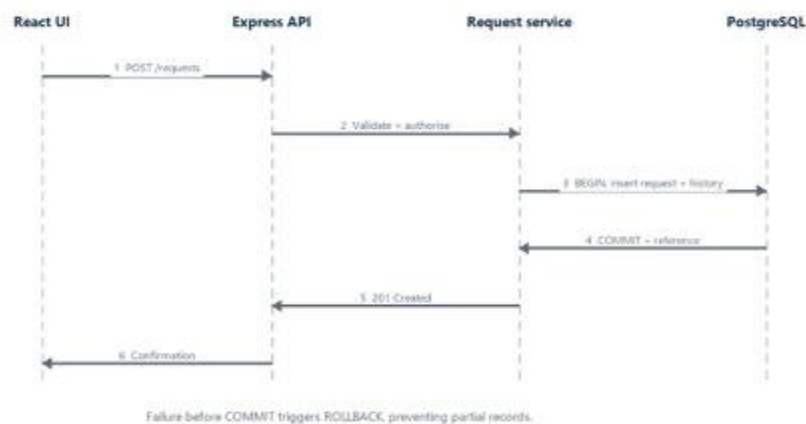


Modules communicate through in-process interfaces and domain events; they do not call another module's tables directly.

Module	Owns	May depend on	Must not do
Identity & Access	Authentication state, users, roles, permissions	Shared security primitives	Expose password/session secrets or embed feature rules
Request Management	Request creation, details, reference number, requester view	Identity, category contract, audit event contract	Write workflow or audit tables directly
Workflow	Assignment, status transitions, staff actions	Identity, request contract, audit event contract	Permit invalid transitions or bypass authorisation
Audit & History	Immutable actor/action/record/time entries; timeline queries	Identity identifiers and domain-event contracts	Mutate prior events through ordinary application paths
Reporting	Read models, counts, ageing and trends	Approved read interfaces/views	Become the source of truth for operational records
Administration	Users, roles, categories and routing configuration	Identity and request configuration contracts	Change live data without audit capture

5.4 Integrity-critical request creation

Figure 4. Request creation integrity sequence



The request row and first history record form one atomic unit of work.

A PostgreSQL transaction treats the operations as one unit: either all changes are committed or none are retained. Primary/foreign keys and uniqueness constraints enforce relationships and prevent duplicate identifiers at the database boundary (Douglas and Douglas, 2005).

5.5 Architecture rules

- React components call the backend only through documented HTTP endpoints; they never connect directly to PostgreSQL.
- Express routes perform transport concerns and delegate business decisions to services; routes do not contain persistence logic.
- Every protected use case checks authenticated identity and permission before reading or changing restricted data.
- A module accesses its own data through its repository or service interface. Cross-module table access is prohibited.
- Integrity-critical changes and their mandatory audit entry share one database transaction.
- Domain events remain in-process in M2. An external broker requires a new ADR and an outbox/retry design.
- Reporting uses indexed, paginated queries or database views and may not weaken the transactional source of truth.
- Runtime secrets are supplied through protected configuration; they are never committed to the repository.

6. Technology stack decision

6.1 Stack fit and proportionality

Layer	Selected technology	Alternatives considered	Decision basis and constraints
-------	---------------------	-------------------------	--------------------------------

Frontend	React	Server-rendered templates; alternative React-like frameworks/meta-frameworks	Chosen team stack for a responsive, interactive UI. Bundle size and initial rendering must be measured because a baseline React SPA can ship more client-side JavaScript; optimisation is evidence-driven (Lonka, 2023).
Backend	Node.js + Express	PHP backend; multiple independently deployed services	Node.js performed better than PHP in the supplied benchmark study. Express keeps the HTTP layer small; business rules remain framework-independent (Odeniran, Wimmer and Rebman, 2023).
Database	PostgreSQL	Document-oriented NoSQL database	CivicConnect has strongly related users, requests, assignments, statuses and audit records. Relational integrity and transaction consistency match NFR-007/008; NoSQL flexibility is not the dominant need (Sahatqija et al., 2018).
Architecture	Layered modular monolith	Unstructured monolith; microservices	Provides one deployment and transaction boundary with maintainable internal seams. Microservices are deferred until measurable independent-scaling or deployment needs exist (Blinowski, Ojdowska and Przybyłek, 2022).

Using JavaScript across the React and Node.js layers can reduce language switching for the three-person team, but it does not remove the need for framework, database and security learning. RISK-001 therefore remains open until small proofs of concept demonstrate authentication, transaction handling, deployment and CI within the M2 schedule (CivicConnect Team, 2026a).

6.2 Version and compatibility control

The research evidence supports technology categories, not current package versions. Exact versions must therefore be fixed from the team's implemented repository rather than invented in this document. The following evidence is required before the PED v2.0 baseline is signed:

Item	Required baseline evidence	Status
Node.js runtime	Exact runtime version in deployment configuration and CI log	Planned
Express, React and supporting packages	package.json plus lockfile committed on the reviewed branch	Planned
PostgreSQL server and driver	Deployment configuration plus successful migration/connection evidence	Planned
Browser compatibility	Test record for latest two Chrome, Edge and Firefox versions at 360/768/1280px	Planned
Hosting compatibility	Repeatable deployment instructions, health check and rollback evidence	Planned

7. Architecture Decision Records

7.1 ADR-ARCH-001 - Select a layered modular monolith

Field	Record
Status	Proposed for team approval; target status: Accepted in PED v2.0
Context	CivicConnect has a three-person team, limited schedule/cost, a relational workflow, strong integrity/audit needs and a modest initial workload. M1 deferred architecture to M2.
Options	Unstructured monolith; layered modular monolith; microservices.
Decision	Use one Node.js/Express deployable with explicit layers and feature modules. Use in-process interfaces/events between modules.
Rationale	Strong transaction boundary and simple deployment fit the constraints; module boundaries support maintainability and an extraction path.
Consequences	Independent module deployment is unavailable initially. Boundary discipline and architecture tests are required to prevent a “big ball of mud”.
Revisit trigger	A module shows sustained, measured scaling pressure; releases are blocked by coupling; or ownership grows beyond the current team.
Linked evidence	ASR-01-07; RISK-001; CON-001-003/008; architecture comparison; RTM rows in Section 8.

7.2 ADR-TECH-001 - Select React, Node.js/Express and PostgreSQL

Field	Record
Status	Proposed for team approval; target status: Accepted in PED v2.0
Context	The chosen stack must support a responsive browser UI, measurable API performance, relational integrity, auditability, low cost and a small team.
Options	React or server-rendered UI; Node.js/Express or PHP backend; PostgreSQL or document-oriented NoSQL.
Decision	Use React in the browser, Node.js with Express for the API and PostgreSQL for persistent data.
Rationale	The supplied evidence supports Node.js performance for the compared workloads, while relational transactions and constraints fit the request workflow and audit model.
Consequences	React bundle and rendering performance must be measured; Node.js CPU-heavy work must not block request handling; database schema migrations require review.
Version control	Exact supported versions are recorded in manifests/lockfiles and deployment evidence before baseline sign-off.
Linked evidence	NFR-003/007/008/012; RISK-001; Odeniran, Wimmer and Rebman (2023); Sahatqija et al. (2018); Douglas and Douglas (2005).

7.3 ADR-MOD-001 - Enforce feature-aligned module boundaries

Field	Record
Status	Proposed for team approval
Context	A monolith without boundaries can accumulate technical debt and scatter security, validation and persistence logic.
Decision	Adopt the modules in Figure 3 and permit interaction only through exported application interfaces or in-process domain events.
Enforcement	Folder/module ownership; dependency rule checks; code review checklist; unit/service/API test separation.
Consequences	Some interfaces appear more formal than direct calls, but changes become easier to locate, test and review.
Linked evidence	NFR-009/012; CON-001/007; Skiada et al. (2018); Koziolok (2011).

8. RTM evolution and evidence plan

The M1 traceability chain is extended from stakeholder need and requirement into ASR, component, ADR and verifiable evidence. Current evidence is labelled honestly: research and design records can be complete while code-level evidence remains planned.

Requirement	ASR / component	Architecture decision	Acceptance / verification evidence	Current status
FR-001 Authenticate users	ASR-01; Identity & Access	ADR-ARCH-001 / ADR-MOD-001	Automated valid/invalid/session tests; unauthorised endpoint suite	Planned
FR-002 Enforce roles	ASR-01; Identity & Access + API middleware	ADR-MOD-001	Role-permission matrix; 100% prohibited cases return 403	Planned
FR-011 Submit request	ASR-02/03; Request + Audit	ADR-TECH-001	Integration test proves request and initial event commit together	Planned
FR-015 Update assignment/status	ASR-01/02/03; Workflow	ADR-MOD-001	Transition, concurrency, permission and audit tests	Planned
FR-017 View history	ASR-03; Audit & History	ADR-ARCH-001	Timeline test with actor/action/record/time for each event	Planned
NFR-003 Performance	ASR-04; UI/API/Reporting/DB	ADR-ARCH-001 / ADR-TECH-001	10,000 records; 50 concurrent users; p95 thresholds	Planned
NFR-007 Integrity	ASR-02; PostgreSQL + services	ADR-TECH-001	Rollback, unique-reference and foreign-key tests	Planned
NFR-008 Auditability	ASR-03; Audit module	ADR-MOD-001	Required-event coverage report; tamper-path review	Planned
NFR-011 Recovery	ASR-06; PostgreSQL/backup	ADR-TECH-001	Backup log and timed restore rehearsal meeting RPO/RTO	Planned

NFR-012 Maintainability	ASR-05; all modules/CI	ADR-MOD-001	PR approvals, automated checks and dependency-rule result	Planned
----------------------------	------------------------	-------------	---	---------

8.1 End-to-end trace example: request creation

Trace step	Evidence / decision
Stakeholder need	Community member needs one reliable channel and a reference for follow-up.
Requirement	FR-011 Submit Request; NFR-007 Integrity; NFR-008 Auditability.
ASR	ASR-02 requires atomic persistence; ASR-03 requires complete event history.
Architecture	React form → Express API → Request service → PostgreSQL transaction; audit entry in the same unit of work.
ADR	ADR-ARCH-001 and ADR-TECH-001.
Verification	API/integration test creates request + initial history; injected failure proves rollback; uniqueness test proves no duplicate reference.
Repository evidence to link	Planned: route/service test path, migration containing constraints, CI run and pull request.

9. Risks, trade-offs and forward engineering

Risk / trade-off	Ripple effect	Control / next evidence	Owner / status
RISK-001 stack unfamiliarity	Learning delays could consume architecture and test time	Time-box authentication, transaction, React and deployment PoCs; record results in ADR-TECH-001	Whole team / Open
Architecture erosion	Direct imports and table access could turn modular monolith into an unstructured monolith	Dependency rule, module ownership and review checklist	Member 1 / Planned
Single database/runtime failure	Simple deployment creates concentrated availability risk	Health monitoring, rollback, daily backup and timed restore rehearsal	Deployment owner / Planned
React initial-load performance	Large client bundle can weaken mobile experience	Bundle measurement, code splitting, pagination and measured rendering strategy	Frontend owner / Planned
Node.js CPU-bound work	Heavy synchronous processing can delay unrelated requests	Keep handlers non-blocking; profile reporting; move genuinely heavy work to controlled background execution only if measured	Backend owner / Planned
Audit data growth	Complete event history increases storage and query cost	Indexes, pagination, retention decision and query plan review	Data owner / Planned
Deferred microservices	Coarse deployment prevents independent scaling	Maintain module seams; revisit ADR only on measured trigger	Architect / Accepted trade-off

Software architecture strongly influences how quickly and correctly a system can be understood, changed, tested and maintained over its life. Architecture evaluation should therefore use concrete change scenarios and measurable evidence rather than rely only on initial implementation speed (Koziolek, 2011).

Distributed web-service auditing must correlate activity across heterogeneous components and workflows. Keeping CivicConnect as one deployable application reduces that correlation problem for M2, although audit records still require complete actor, action, record and time context (Bates et al., 2017).

9.1 Deliberate deferrals

Deferred item	Why it is deferred	Trigger to reconsider
Microservices	No demonstrated independent scaling/deployment need; overhead conflicts with team, schedule and cost constraints	Measured bottleneck or release/ownership independence that cannot be solved proportionately
External message broker	In-process events meet current use cases and keep transaction/error handling visible	Reliable cross-system asynchronous integration becomes in-scope
Separate reporting database	10,000-record target does not justify replication and consistency overhead	Production query load harms transactional service despite indexing/optimisation
Server-side React rendering/meta-framework	CivicConnect is an authenticated operational app, not a public content/SEO product; extra server/build complexity is not yet justified	Measured initial-render failure against the agreed performance target
External notifications	Explicitly deferred in M1 scope baseline	Approved scope change with stakeholder value, privacy and delivery evidence

10. Data Entities & Ownership

The following entities were derived directly from the M1-baselined functional requirements (FR-002 to FR-018) rather than invented independently. Each entity's ownership and lifecycle are traced to the requirement that created the need for it.

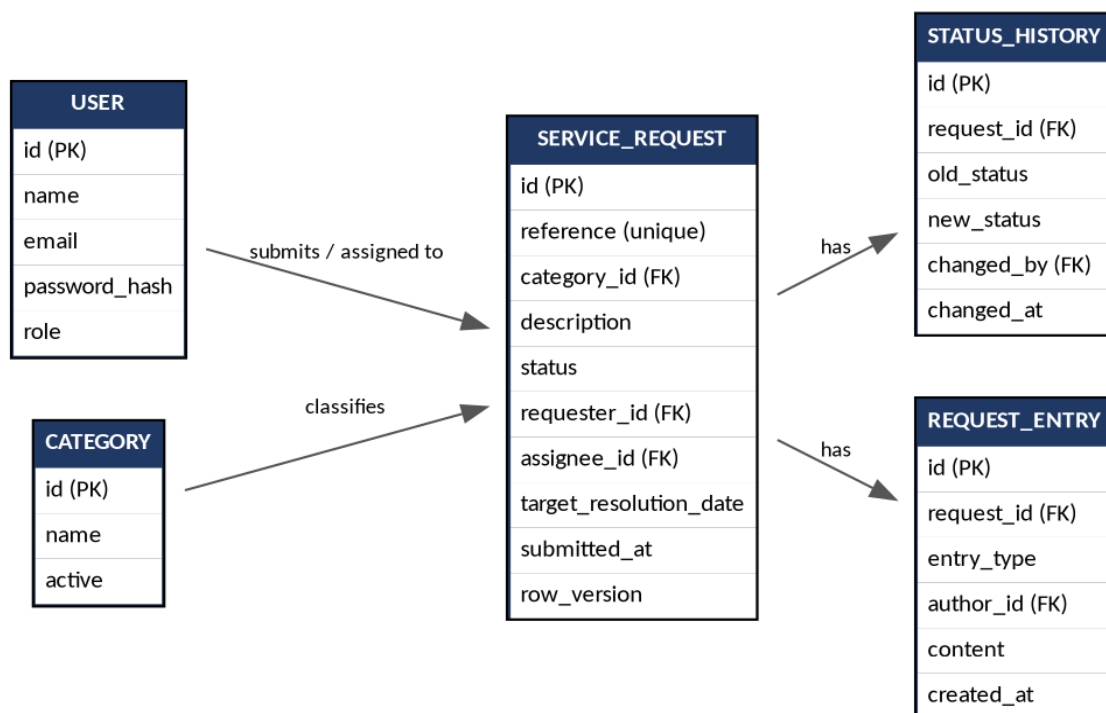
Entity	Key attributes	Owned/created by	Notes
User	id, name, email, password hash, role	Admin (FR-018) / self-registration for Requesters	Role-based access per NFR-001
ServiceRequest	id, reference (unique), category, description, status, submitted_at, target_resolution_date, requester_id, assignee_id	Requester creates (FR-002); Staff updates (FR-010, FR-011)	Reference must never duplicate (NFR-007)
Category	id, name, active flag	Admin (FR-018)	Fixed set of 7 + "Other" (RD-03)

Entity	Key attributes	Owned/created by	Notes
StatusHistory	id, request_id, old_status, new_status, changed_by, changed_at	System, on every transition	This is the audit trail (NFR-008)
RequestEntry	id, request_id, type (Public/Internal), author_id, content, created_at	Staff (FR-012, FR-013)	Public/internal split is a hard access boundary

ServiceRequest is the aggregate root; StatusHistory and RequestEntry are child records that must never outlive or be orphaned from their parent request - this is what makes referential integrity the actual requirement here, not just "having a database."

11. Initial Data Model (ERD)

Proportionate to the M1 baseline - no premature normalisation beyond what NFR-007 (integrity) and the public/internal split (FR-012) actually require. Assignment (who is responsible for a request, and by when) is folded into ServiceRequest rather than a separate table, since M1 never treats "assigned" as anything beyond a request attribute.



12. ADR-M2-DATA-001: Persistence Model

Status: Approved - 26 September 2026

CivicConnect's data is structurally relational: Request, User, Category, StatusHistory and RequestEntry all have fixed, well-defined relationships. NFR-007 (no duplicate references, no partial transactions) and NFR-008 (complete, attributable audit trail) both require atomic, consistent writes across related tables - a status change and its history entry must commit together or not at all.

Alternatives considered

Option	Why considered	Decision
Relational (PostgreSQL)	Structured entities, foreign-key relationships, ACID transactions	Accepted
Document store (MongoDB)	Some team exposure via other modules	Rejected - NFR-007's atomicity requirement is awkward across document collections without relational transaction guarantees; no unstructured data exists to justify the trade-off
Hybrid (relational + cache)	Could pre-empt performance concerns	Rejected for M2 - NFR-003's targets are easily met by a single well-indexed PostgreSQL instance at this scale

Decision

PostgreSQL as the sole persistence store for M2.

Rationale

Matches the actual shape of CivicConnect's data and provides transactional integrity guarantees natively rather than requiring application-level enforcement. Free and open-source (CON-003). See A2 Task 2 research (Persistence & Data Integrity) for the deeper comparison of transaction isolation levels - not duplicated here per the M2 brief's instruction not to copy A2 research into the PED.

Integrity, concurrency and risk measures

- Uniqueness: reference has a database-level UNIQUE constraint - duplicates are rejected by the database, not just the application (NFR-007).
- Concurrency: optimistic locking via a row_version column on service_requests, checked on every write - chosen over long-held locks given the small concurrent-user count (NFR-003: 50).
- Auditability: status_history and request_entries are append-only by design (NFR-008).
- Backup/recovery (Should, NFR-010/011): Supabase's managed Postgres includes automated backups; a documented restore procedure satisfies the M2 expectation.
- SPOF: a single database instance is a single point of failure - accepted as proportionate risk at M2 scale (CON-001/CON-003) and recorded in the Risk Register (RISK-008).

Trade-offs

Less flexibility for unstructured/evolving data shapes than a document store - accepted because CivicConnect's entities are stable and well understood from the M1 baseline.

Later consequence

If a future requirement introduced genuinely unstructured or highly variable data, this decision would need to be revisited through formal change control per CON-009.

13. ADR-M2-TECH-002: Technology Stack Selection

Status: Approved by team (Jininy Nkomo, Matamela Davhana, Mkhanyisi Mqadi)

Date confirmed: 26 September 2026

Owner: Jininy Nkomo

Context

RD-10 (M1 PED) deliberately deferred all technology decisions to M2. The stack must support NFR-003 (10,000 requests, 50 concurrent users, sub-2s operations), NFR-005 (WCAG 2.2 AA), NFR-006 (responsive at 360/768/1280px), and must be realistically learnable by all three team members within the schedule (CON-001, CON-002, RISK-001 - technology unfamiliarity already flagged critical in M1).

Alternatives considered

Criterion	Node.js + Express + React	Java + Spring Boot
Team capability	JS/Node already used across the team's other portfolio work	Strong for one member; unconfirmed for the other two at proposal time
Learning curve (RISK-001)	Lower - one language front-to-back	Higher - separate language/paradigm for the frontend
Cost (CON-003)	Free/open-source; free-tier hosting widely available	Free/open-source; free-tier hosting less common for Java web apps
Deployment compatibility	Straightforward containerisation; fits common free-tier PaaS offerings	Requires more memory/startup time, less free-tier friendly

Decision

- Backend: Node.js + Express
- Frontend: React
- Database: PostgreSQL (see ADR-M2-DATA-001), hosted on Supabase (free tier)
- Backend hosting: Render (free web service tier)
- Data access: raw parameterised SQL via node-postgres (pg), with node-pg-migrate for schema migrations - not a full ORM

Rationale

Directly addresses RISK-001 by minimising the number of new languages and paradigms the team must learn simultaneously, while meeting every relevant NFR. Java + Spring Boot was genuinely viable (fits the team's existing PRG381 inventory-system experience) but introduces more moving parts for a three-person team on an academic schedule. Raw SQL over an ORM keeps transaction control explicit (needed for the atomic status/history writes required by NFR-007) and avoids adding a second unfamiliar abstraction layer.

Hosting decision and documented free-tier limits (CON-003)

Service	Free-tier limit	Why it matters here
Render (web service)	750 instance-hours/month; spins down after 15 min inactivity (~1 min cold start); 512MB RAM / 0.1 vCPU	Acceptable for a demo/academic-scale deployment; cold start is a known, documented limitation
Render (Postgres, free)	Rejected - 1GB storage but hard-deletes the database 30 days after creation (14-day grace period)	Unsuitable for a semester-long project; data would be lost mid-project
Supabase (Postgres, free)	500MB storage; project pauses (not deleted) after 7 days of inactivity, restorable from the dashboard	Selected over Render's own Postgres specifically to avoid the 30-day hard-deletion risk

Trade-offs

Express provides less built-in structure than Spring Boot, so the team must be deliberate about layering (routes → services → data access) - addressed by the Member 3 design-pattern decisions. Supabase's pause-after-inactivity behaviour means the team must occasionally "wake" the project during low-activity periods.

Versions (confirmed, as scaffolded)

Component	Version
Node.js	v26.3.0 (Current release; enters Active LTS October 2026 - Node 24 remains the established LTS if a more conservative choice is preferred)
Express	5.2.1
React	19.3.0
PostgreSQL (Supabase-provisioned)	17.6
node-pg-migrate	9.0.0
pg (PostgreSQL driver)	8.23.0

Later consequence

If the free-tier limits above (500MB storage, 750 instance-hours) are exceeded as the project grows, this decision must be revisited through formal change control (CON-009) - likely a move to a paid tier or a different provider, documented with updated cost evidence.

14. RTM Updates - Data/Persistence & Technology Columns

Requirement	Data/persistence impact	Technology decision	Status
FR-002	Creates ServiceRequest row; unique reference enforced at DB level	Express route + PostgreSQL insert	In Development
FR-004, FR-005	Read from ServiceRequest filtered by requester_id	React view + Express query, ownership check at query level	Planned
FR-009	Full detail join across ServiceRequest, StatusHistory, RequestEntry	Express endpoint with role check	Planned
FR-010, FR-011	Writes to ServiceRequest + insert into StatusHistory; one transaction	PostgreSQL transaction (BEGIN/COMMIT)	Planned
FR-012	RequestEntry.entry_type filters what Requesters/Management see	Query-level filter, never application-only	Planned
FR-015	Aggregated read excluding description/requester identity/internal comments	Separate management-view query, not a filtered full record	Planned
NFR-003	Indexing on status, category_id, target_resolution_date	PostgreSQL indexes	Planned
NFR-007	Unique constraint on reference; FK constraints; transactions	PostgreSQL constraints	Planned
NFR-008	Append-only StatusHistory/RequestEntry - no UPDATE/DELETE at app layer	PostgreSQL role permissions	Planned
NFR-010, NFR-011	N/A (hosting-level)	Managed Postgres provider with automated daily backup	Planned

15. Risk Register Additions

ID	Risk	Probability/Impact	Mitigation	Owner
RISK-006	Chosen query library unfamiliar to team, slowing development	Medium/Medium	Keep to raw parameterised SQL rather than a full ORM for M2	Jininy Nkomo
RISK-007	Free-tier PostgreSQL hosting has connection/storage limits that could be hit later	Low/Medium	Documented provider free-tier limits in the ADR now; revisit before M3 if usage grows	Jininy Nkomo
RISK-008	Single database instance is a single point of failure	Low/Medium (accepted for M2 scale)	Documented and deferred - no redundancy required at this milestone	Jininy Nkomo

16. Deployment Compatibility (Section 5.8)

- Direction: Node/Express backend + PostgreSQL on a free-tier PaaS (Render), React frontend served separately or from the same origin.

- Config/secrets: DB connection string and any auth secret via environment variables, never committed.
- Networking: cross-origin frontend/backend calls require explicit CORS configuration.
- Deferred: load balancing, multi-region, and horizontal scaling - explicitly out of scope at M2, recorded as forward engineering considerations.

17. Design Decisions

17.1 Design Problem 1: Request Type Handling

Context:

CivicConnect must handle multiple request categories, such as maintenance, IT support, and safety, each with distinct validation and prioritization rules. A single monolithic request-creation class would violate SRP and OCP, making the system hard to extend and test.

A2 Evidence:

Assignment 2 compared Factory method and Strategy patterns.

- Factory method focuses on object creation and would require new subclasses for every category.
- Strategy encapsulates variable behaviour, allowing category-specific logic to vary independently of the class.

Final decision:

Adopt the Strategy pattern for category-specific validation and priority logic.

Why it Fits CivicConnect:

- Aligns with the SRP, OCP, and DIP principles.
- Keeps the request class cohesive and testable.
- Categories share the underlying request structure whereas variation lies in rules, not object shape.
- Supports future expansion without modifying existing code.

Expected Benefits:

- Reduced coupling between request logic and category rules.
- Improved cohesion and maintainability.
- Easier unit testing in individual strategies.

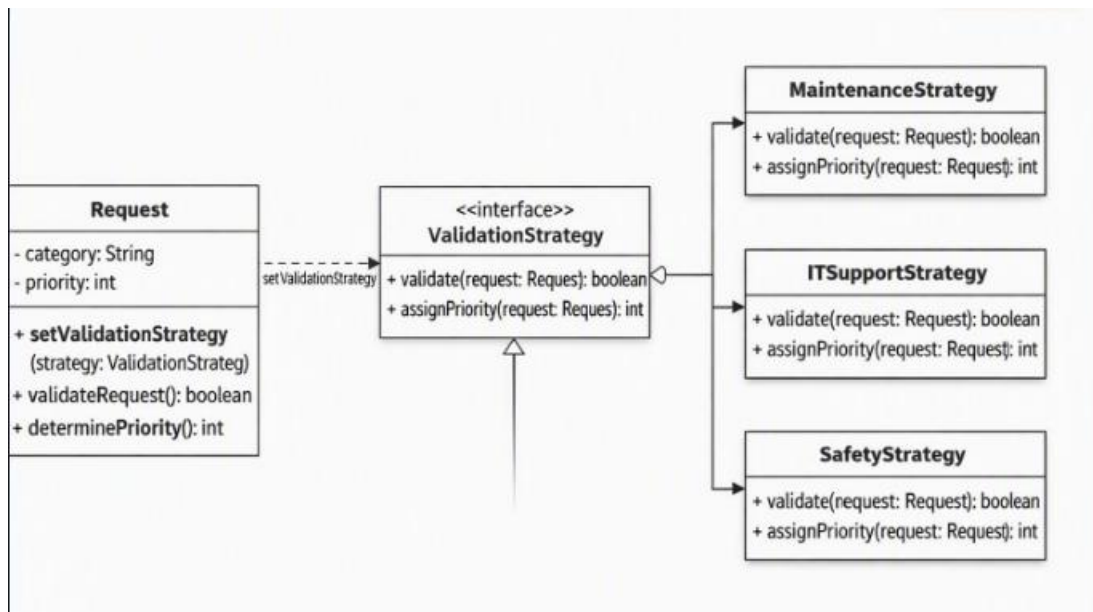
Trade-offs:

- More classes to maintain.
- Slightly higher abstraction overhead.

Affected Modules/Classes:

Request, ValidateStrategy interface, concrete strategies (MaintainabilityStrategy, ITsupportStrategy, SafetyStrategy).

Design Diagram:



Implementation Evidence:

Implementation has not yet begun. This decision is recorded as forward engineering and will be implemented in Milestone 3.

17.2 Design Problem 2: Status Change Notification Coupling

Context:

When a request's status changes, multiple actions occur. Notifying citizens, updating dashboards, and recording audit logs. Direct calls to each service create tight coupling and violate SRP and DIP, making the system fragile and hard to expand.

A2 Evidence:

Assignment 2 compared Observer and Mediator patterns.

- Observer supports independent, order insensitive reactions.
- Mediator centralizes coordination but risks becoming a single-point-of-error.

Final Decision:

Adopt the Observer pattern for status-change notifications.

Why it Fits ClivicConnect:

- Decouples RequestStatus from concrete notification channels.
- Each observer (Email, SMS, Dashboard, Audit) handles one responsibility.
- New channels can be added without modifying the subject.

- Aligns with FR-011, FR-014, and NFR-008 (auditability).

Expected Benefits:

- Reduced coupling and improved cohesion.
- Simplified extensibility for future notification channels.
- Clear separation of responsibilities.

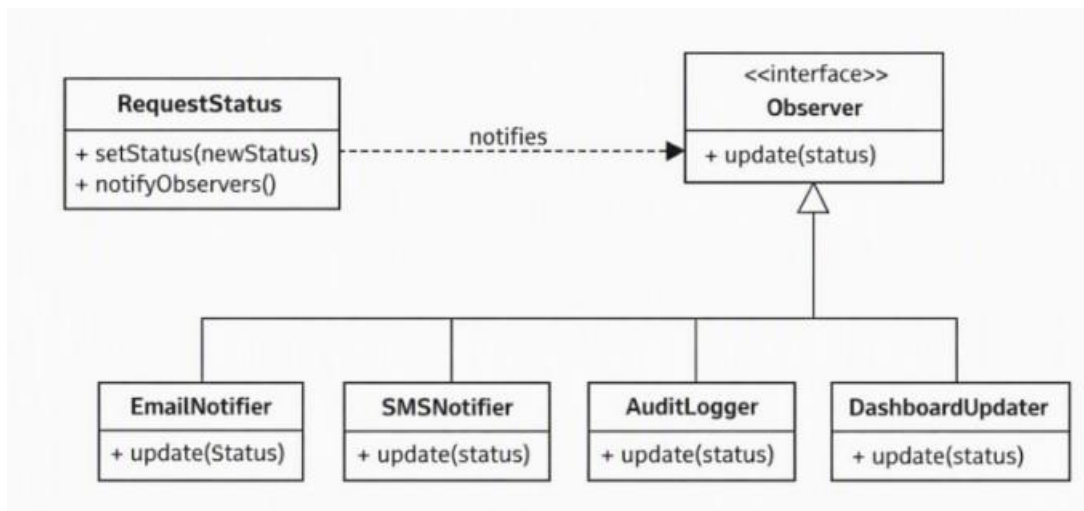
Trade-offs:

- Implicit notification ordering. May require mediator if coordination becomes complex.
- Slight increase in setup complexity.

Affected Modules/Classes:

RequestStatus (subject), Observer interface, concrete observers (EmailNotifier, SMSNotifier, AuditLogger, DashboardUpdater).

Design Diagram:



Implementation Evidence:

Implementation has not yet begun. This decision is recorded as forward engineering and will be implemented in Milestone 3.

17.3 Engineering Decision Log (ADR)

Decision ID	Context	Constraints	Alternatives	Decision	Rationale
DEC-001	Repository structure and governance	CON-007 requires controlled repo, PRs, and two approvals.	1.Single repo with protected main. 2.Multi-repo modular. 3.Informal repo.	Adopt single team repository with protected main branch.	Simpler governance, easier traceability, and aligns with mandatory controls.
DEC-002	Requester feedback mechanism	CON-003, cost, CON-006, security, and FR-006.	1.In-system timeline.	Provide feedback via in-system timeline only.	Meets minimum stakeholder need

			2.Email/SMS notifications. 3.WhatsApp push.		without external cost or privacy risk.
DEC-003	Management access model	CON-006, security, NFR-002, and FR-015.	1.Full access to all request details. 2. Metadata only. 3. Role-based split.	Provide metadata summaries only while the sensitive details are excluded.	Reduces privacy exposure while meeting oversight needs.
DEC-004	Request-type handling logic	CON-005, quality, FR-002, and FR-003.	1.Factory method. 2. Strategy 3.Hard-coded conditions.	Adopt strategy pattern for category-specific validation and priority rules.	Encapsulates variable behaviour, reduces coupling, improves cohesion, and testability. Aligns with SRP and OCP.
DEC-005	Status-change notification coupling.	CON-006, security, FR-011, FR-014, and NFR-008.	1.Observer 2. Mediator 3. Direct Calls	Adopt observer pattern for independent notification channels.	Decouples status logic from concrete notifiers, improves extensibility, and auditability. Aligns with DIP and SRP.
DEC-006	Status update, audit/history integration	FR-017, NFR-008, CON-006, and CON-010.	1.REST API 2.In-process 3.Asynchronous messaging.	Publish/Subscribe asynchronous messaging.	Decouples modules, resilient, extensible.

Decision ID	Trade-offs	Risks	Evidence	Later Consequence	Later Consequence Update
DEC-001	Less modular separation with scaling issues if project grows.	Merge conflicts and reduced flexibility.	Maser Brief governance rules and team agreement.	Will require stricter branching discipline in M3 and may need modularisation if scope expands.	M2:Governance confirmed.
DEC-002	Requesters must log in to see updates.	Reduced convenience for requesters.	FR-006 acceptance criteria and stakeholder analysis.	External notification deferred until cost and privacy evidence available.	M2: Timeline prototyped.
DEC-003	Slightly less convenient for management drill down.	Risk of management dissatisfaction.	FR-015 acceptance criteria and stakeholder conflict resolution.	Management must request Staff role if deeper access needed.	M2:decision baselined.
DEC-004	More classes to maintain.	Over engineering if rules are trivial.	UML diagram and strategy classes	Easy extension of new categories.	M2: decision baselined.
Dec-005	Implicit ordering	Notification ordering issue.	UML diagram and observer classes	New channels can be added without subject change.	M2:decision baselined.
DEC-006	Requires messaging infrastructure.	Complexity of event schema.	Forward Engineering. Planned for M3.	Event publisher/subscriber to be implemented.	M2: Updated once code is committed in M3.

(Continued)

18. Deployment Compatibility

Current Deployment Direction / Environment Assumption

The system is expected to be initially deployed on a university-managed server environment, running Node.js/Express for backend services, React for the frontend, and PostgreSQL for persistence. This environment is assumed to be Windows Server or Linux-based with internal LAN access.

Architecture / Technology Plausibility

The selected stack is fully compatible with the assumed environment. Using JavaScript end-to-end (Node.js + React) lowers the RISK-001 learning-curve risk, while PostgreSQL provides strong relational data support. This combination is plausible for both on-premise and cloud deployment (Azure, Vercel, Fly.io).

Configuration, Secrets, Persistence, Networking Implications

- Configuration handled via environment variables (.env files).
- Secrets (DB credentials, API keys) must be stored securely, not hardcoded.
- Persistence managed through PostgreSQL with transaction boundaries defined.
- Networking limited to internal LAN initially, with firewall rules required if external APIs are exposed later.

Material Compatibility, Cost, Availability, Operational Risks

- Compatibility: Node.js, React, and PostgreSQL are widely supported and available.
- Cost: Minimal for internal server; cloud scaling introduces subscription costs.
- Availability: Dependent on university IT infrastructure; cloud deployment increases resilience.
- Risks: Misconfigured secrets, insufficient server resources, dependency on external providers if cloud is adopted.

Deferred Deployment Decisions

- Cloud scaling (Azure, Vercel, Fly.io) deliberately deferred until performance/load testing evidence is available.
- Containerization (Docker/Kubernetes) deferred until operational need is proven.
- Evidence required later: performance metrics, user load analysis, IT governance approval.

19. Appendix A AI Usage Register Entry

Date	Member	Tool	Engineering task	AI contribution	Verification	Decision	Issues found
7 Sept 2026	Mkhanyisi Mqadi	OpenAI Codex	Requirements , constraints and RTM drafting	Proposed requirement wording, acceptance measures, requirement-level decisions, constraint analysis and RTM structure	Compared to the Master Project Brief, Milestone 1 brief and Member 1 stakeholder/scope analysis; team must review all targets before baseline	Use after human review and controlled peer approval	AI initially required project-specific source material; architecture and technology choices were deliberately not invented
8 September 2026	Matamela Davhana	Copilot	Risk Register and Engineering Decision Log drafting	Suggested risks, probability or impact rating, and mitigation strategies. Suggested ADR format and rationale.	Verified against Master Brief risk standards. Reviewed against CON-007 governance constraint.	Accepted by team after review and edits.	Phrasing adjusted for clarity and needed to change
7 September 2026	Jininy Nkomo	Claude	Problem And Business Need, Stakeholder Analysis, Scope Baseline drafting	Used to help with research of the problem, stakeholders, stakeholder conflicts and scope. Also used it to check whether the sections were clear and whether there were any missed important areas from the Milestone 1 requirements.	Compared against Master Brief and Milestone 1 brief scenario details; rejected AI-proposed numeric threshold not backed by team evidence.	Used after human review and controlled peer approval	AI was mainly used for structuring, wording, and reviewing. Unsupported AI suggestions were removed before the baseline.
30 September 2026	Matamela Davhana	Copilot	Design Decision Produce initial design diagrams from source given.	Generated draft diagram based on ADRs and requirements.	Reviewed against A2 research, traced to RTM, and checked for consistency with chosen tech stack.	Accepted as draft visualization and recorded in PED as forward engineering evidence.	Diagrams lacked clear data flow and had missing modules. Required refinement of query and validation risk of superficial representation if not cross-checked.
30 September 2026	Jininy Nkomo	Claude	Data, persistence and technology documentation	Used to support research, clarification and refinement of data entities, persistence decisions, technology-stack	Compared the suggestions against the Master Project Brief, Milestone 2 requirements and the existing CivicConnect	Used after human review and controlled peer approval.	AI suggestions were treated as proposals rather than final decisions; unsupported assumptions and technical claims were

				documentation, ADRs, risk considerations and RTM entries.	baseline/repository evidence. Unsupported or unverified technical claims were reviewed and corrected before inclusion.		not retained without project evidence.
--	--	--	--	---	--	--	--